Toy Blockchain Research Report

Project: Toy Blockchain in Go **Author:** Shanilka Thulshani **Language:** Go **Assessment:** Golang Backend Developer Assessment

---

1. Introduction

This report presents the implementation and evaluation of a toy blockchain developed in Go. The project demonstrates the core concepts of blockchain technology, including cryptographic hashing, proof-of-work mining, transaction validation, ledger management, blockchain validation, and persistent storage.

Unlike production blockchain systems, this implementation focuses on understanding the internal mechanisms of a blockchain rather than scalability or distributed consensus. Several experiments were conducted to evaluate the behaviour of the implementation, including tamper detection and the impact of mining difficulty on computational effort.

---

2. Tamper-Evidence Experiment

2.1 Objective

The objective of this experiment was to verify that any modification to data stored in the blockchain can be detected through blockchain validation.

2.2 Method

A blockchain containing multiple blocks was created. After the blocks were mined successfully, a transaction inside an earlier block was deliberately modified without recalculating its hash.

The blockchain validation routine was then executed.

2.3 Results

Before Tampering

```
Before tampering:
true
```

After Tampering

```
After tampering:
Detected: block 1 hash mismatch:
stored=00e027eab64ac7ad80dc83707810047594dc6c34df09f09a303c8a237b0b21c1
calculated=d9bd43b1cc1bd888cada72678e0cc65ee4cbf0d28c6549def6d430392629523a
```

2.4 Discussion

Each block stores a SHA-256 hash calculated from its contents, including the block index, timestamp, transactions, previous hash, and nonce.

When a transaction inside an already mined block was modified, the stored hash no longer matched the newly calculated hash. During validation, the blockchain recomputed the hash for every block and compared it with the stored value.

The validation process detected the inconsistency immediately and reported the first invalid block. This demonstrates the tamper-evident property of blockchains, where any modification to historical data becomes detectable.

---

3. Mining Difficulty versus Computational Effort

3.1 Objective

This experiment investigated how increasing the proof-of-work difficulty affects mining effort.

3.2 Method

The same block was mined multiple times while changing only the mining difficulty. The implementation recorded the number of hashing attempts required and the total mining duration.

3.3 Results

Difficulty	Hash Attempts	Mining Time
1	14	0 s
2	47	0 s
3	11,031	17.48 ms
4	51,179	41.37 ms

3.4 Analysis

The experimental results show that mining effort increases rapidly as difficulty increases.

The increase is not linear. Each additional leading hexadecimal zero significantly reduces the probability that a randomly generated hash satisfies the mining target. Consequently, miners must perform many more SHA-256 calculations before finding a valid nonce.

Although the measured times remain relatively small because this project mines only a few blocks on a local computer, the number of required hash attempts increases dramatically. This behaviour reflects the exponential nature of proof-of-work systems used in real-world blockchains.

---

4. Design Write-up

4.1 Block Structure

Each block in the blockchain contains the following fields:

- Block index
- Unix timestamp
- List of transactions
- Previous block hash
- Nonce
- Current block hash

The genesis block is deterministic and always begins the blockchain with a fixed previous hash consisting entirely of zeros.

4.2 Hashing Scheme

The project uses the SHA-256 cryptographic hash function to generate block hashes.

Before hashing, block data is converted into a deterministic binary representation using length-prefixed encoding. This avoids ambiguity that can occur with simple string concatenation or delimiter-based serialization.

The fields included in the hash calculation are serialized in the following order:

1. Block index (binary encoded integer)
2. Timestamp (Unix timestamp)
3. Number of transactions
4. Transaction details:
    - Sender (length-prefixed string)
    - Recipient (length-prefixed string)
    - Amount (binary encoded floating-point value)
5. Previous block hash (length-prefixed string)
6. Nonce (binary encoded integer)

The block's own hash field is intentionally excluded from the calculation because it is the value being generated.

Using deterministic binary serialization ensures that identical blocks always produce the same SHA-256 hash while preventing serialization ambiguity and improving the reliability of the hashing process.

4.3 Validation Process

The blockchain validation routine verifies the integrity and consistency of the entire blockchain by performing several checks:

- The stored block hash matches the recalculated SHA-256 hash.
- The previous hash reference matches the hash of the preceding block.
- The block satisfies the configured proof-of-work difficulty requirement.
- Block indexes and ordering remain consistent.
- Timestamps maintain a valid chronological order.
- All transactions are replayed through the ledger to verify account balance consistency.
- Transactions that attempt invalid operations, such as spending more than the available balance, are rejected.

If any validation check fails, the routine immediately reports the first offending block and transaction (if applicable), and the blockchain is considered invalid.

---

5. Discussion Questions

5.1 How does the previous-hash link make tampering with an old block impractical in a real blockchain?

Each block stores the cryptographic hash of the previous block. If an attacker modifies an earlier block, its hash changes immediately. Because every following block references that hash, all subsequent blocks also become invalid.

In a real blockchain, the attacker would have to recompute the proof-of-work for the modified block and every block that follows while simultaneously catching up with honest miners who continue extending the chain. For large public blockchains, this requires an unrealistic amount of computational power, making historical tampering practically infeasible.

5.2 Alternative Consensus Mechanism

One alternative to Proof-of-Work is **Proof-of-Stake (PoS)**.

Advantage

Proof-of-Stake consumes significantly less electrical energy because validators are selected according to their stake instead of repeatedly performing computationally expensive hashing operations.

Drawback

Proof-of-Stake requires additional mechanisms to discourage dishonest behaviour and can concentrate influence among participants with larger financial stakes.

5.3 Differences Between This Toy Blockchain and a Production Blockchain

This implementation differs from production blockchain systems in several important ways:

1. **No distributed consensus**

   The blockchain executes on a single computer without peer-to-peer communication or network consensus.

2. **No digital signatures**

   Transactions are not cryptographically signed, so sender ownership cannot be verified.

3. **No Merkle Tree**

   Transactions are stored directly inside blocks without constructing a Merkle tree for efficient verification.

5.4 Proposed Improvement

One valuable improvement would be the addition of **digital signatures**.

Each user could own a public-private key pair. When creating a transaction, the sender would sign the transaction using their private key. During validation, every node would verify the signature using the sender's public key before accepting the transaction.

This would prevent attackers from creating fraudulent transactions on behalf of other users and would significantly improve the authenticity of the blockchain.

---

6. Conclusion

This project successfully demonstrates the fundamental concepts of blockchain technology through a working implementation in Go.

The experiments confirmed that blockchain data is tamper-evident and that proof-of-work difficulty has a substantial impact on mining effort. The modular architecture, deterministic hashing, ledger validation, proof-of-work mining, and persistent storage provide a solid educational implementation while remaining considerably simpler than production blockchain systems.

Although many advanced blockchain features remain outside the scope of this project, the implementation provides a strong foundation for future extensions such as digital signatures, peer-to-peer networking, Merkle trees, and distributed consensus.

References

1. Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. Available: https://bitcoin.org/bitcoin.pdf

2. Andreas M. Antonopoulos. *Mastering Bitcoin: Programming the Open Blockchain*. 2nd Edition. O'Reilly Media, 2017.

3. Daniel Drescher. *Blockchain Basics: A Non-Technical Introduction in 25 Steps*. Apress, 2017.

4. Go Authors. *The Go Programming Language Documentation*. https://go.dev/doc/